Notebook: Naive_bayes/from_scratch.ipynb

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.naive_bayes import GaussianNB
```

[--- CODE CELL ---]

```
data = load_iris()
```

[--- CODE CELL ---]

```
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")
df = pd.concat([X,y], axis=1)
```

[--- CODE CELL ---]

```
sns.countplot(x="target", data=df)
plt.title("Class Distribution")
plt.show()
```

[--- CODE CELL ---]

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=42)
```

[--- CODE CELL ---]

```
gnb = GaussianNB()
gnb.fit(X_train, y_train)
```

[--- CODE CELL ---]

```
y_pred_nb = gnb.predict(X_test)
```

[--- CODE CELL ---]

```
acc_nb = accuracy_score(y_test, y_pred_nb)
acc_nb
```

[--- CODE CELL ---]

```
def summarize_by_class(X,y):
    summaries = {}
    for cls in np.unique(y):
        X_cls = X[y == cls]
        summaries[cls] = {
        "mean": X_cls.mean(axis=0),
        "var": X_cls.var(axis=0),
        "prior": X_cls.shape[0] / X.shape[0]
        }
    return summaries
```

[--- CODE CELL ---]

```
def gaussian_pdf(x, mean, var):
    eps = 1e-9
    coefficient = 1 / np.sqrt(2 * np.pi * var + eps)
```

```
    exponent = np.exp(- (x - mean) ** 2 / (2 * var + eps))
    return coefficient * exponent
```

[--- CODE CELL ---]

```
def predict_one(x, summaries):
    posteriors = {}
    for cls, stats in summaries.items():
        prior = np.log(stats["prior"])
        likelihood = np.sum(
                np.log(gaussian_pdf(x, stats["mean"], stats["var"]))
        )
        posteriors[cls] = prior + likelihood
    return max(posteriors, key=posteriors.get)
```

[--- CODE CELL ---]

```
def predict(X, summaries):
    return np.array([predict_one(x, summaries) for x in X])
```

[--- CODE CELL ---]

```
summaries = summarize_by_class(
    X_train.values,
    y_train.values
)
y_pred_custom = predict(X_test.values, summaries)
acc_custom = accuracy_score(y_test, y_pred_custom)
acc_custom
```

[--- CODE CELL ---]

```
print("Sklearn GaussianNB Accuracy:", acc_nb)
print("Custom Naive Bayes Accuracy:", acc_custom)
```

[--- CODE CELL ---]